

DataParallel#

<https://pytorch.org/docs/stable/generated/torch.nn.DataParallel.html>

Description:

Implements data parallelism at the module level. This container parallelizes the application of the given module by splitting the input across the specified devices by chunking in the batch dimension (other objects will be copied once per device). In the forward pass, the module is replicated on each device, and each replica handles a portion of the input. During the backwards pass, gradients from each replica are summed into the original module. The batch size should be larger than the number of GPUs used. It is recommended to use `DistributedDataParallel`, instead of this class, to do multi-GPU training, even if there is only a single node. See: Use `nn.parallel.DistributedDataParallel` instead of multiprocessing or `nn.DataParallel` and Distributed Data Parallel.

Function Signature

```
class torch.nn.DataParallel(module, device_ids=None,  
output_device=None, dim=0)[source]#
```

Parameters

Variables

Code Examples

```
>>> net = torch.nn.DataParallel(model, device_ids=[0, 1, 2])  
>>> output = net(input_var) # input_var can be on any device,  
including CPU
```

Notes & Warnings

WARNING

Warning It is recommended to use `DistributedDataParallel`, instead of this class, to do multi-GPU training, even if there is only a single node. See: `Use nn.parallel.DistributedDataParallel instead of multiprocessing or nn.DataParallel and Distributed Data Parallel.`

 WARNING

Warning In each forward, module is replicated on each device, so any updates to the running module in forward will be lost. For example, if module has a counter attribute that is incremented in each forward, it will always stay at the initial value because the update is done on the replicas which are destroyed after forward. However, DataParallel guarantees that the replica on device[0] will have its parameters and buffers sharing storage with the base parallelized module. So in-place updates to the parameters or buffers on device[0] will be recorded. E.g., BatchNorm2d and spectral_norm() rely on this behavior to update the buffers.

 WARNING

Warning Forward and backward hooks defined on module and its submodules will be invoked len(device_ids) times, each with inputs located on a particular device. Particularly, the hooks are only guaranteed to be executed in correct order with respect to operations on corresponding devices. For example, it is not guaranteed that hooks set via register_forward_pre_hook() be executed before all len(device_ids) forward() calls, but that each such hook be executed before the corresponding forward() call of that device.

 WARNING

Warning When module returns a scalar (i.e., 0-dimensional tensor) in forward(), this wrapper will return a vector of length equal to number of devices used in data parallelism, containing the result from each device.

NOTE

Note There is a subtlety in using the pack sequence -> recurrent network -> unpack sequence pattern in a Module wrapped in DataParallel. See My recurrent network doesn't work with data parallelism section in FAQ for details.

Detailed Documentation

Documentation

Implements data parallelism at the module level.

This container parallelizes the application of the given module by splitting the input across the specified devices by chunking in the batch dimension (other objects will be copied once per device). In the forward pass, the module is replicated on each device, and each replica handles a portion of the input. During the backwards pass, gradients from each replica are summed into the original module.

The batch size should be larger than the number of GPUs used.

It is recommended to use DistributedDataParallel, instead of this class, to do multi-GPU training, even if there is only a single node. See: Use nn.parallel.DistributedDataParallel instead of multiprocessing or nn.DataParallel and Distributed Data Parallel.

Arbitrary positional and keyword inputs are allowed to be passed into DataParallel but some types are specially handled. tensors will be scattered on dim specified (default 0). tuple, list and dict types will be shallow copied. The other types will be shared among different threads and can be corrupted if written to in the model's forward pass.

The parallelized module must have its parameters and buffers on `device_ids[0]` before running this `DataParallel` module.

In each forward, module is replicated on each device, so any updates to the running module in forward will be lost. For example, if module has a counter attribute that is incremented in each forward, it will always stay at the initial value because the update is done on the replicas which are destroyed after forward. However, `DataParallel` guarantees that the replica on `device[0]` will have its parameters and buffers sharing storage with the base parallelized module. So in-place updates to the parameters or buffers on `device[0]` will be recorded. E.g., `BatchNorm2d` and `spectral_norm()` rely on this behavior to update the buffers.

Forward and backward hooks defined on module and its submodules will be invoked `len(device_ids)` times, each with inputs located on a particular device. Particularly, the hooks are only guaranteed to be executed in correct order with respect to operations on corresponding devices. For example, it is not guaranteed that hooks set via `register_forward_pre_hook()` be executed before all `len(device_ids)` `forward()` calls, but that each such hook be executed before the corresponding `forward()` call of that device.

When module returns a scalar (i.e., 0-dimensional tensor) in `forward()`, this wrapper will return a vector of length equal to number of devices used in data parallelism, containing the result from each device.

There is a subtlety in using the pack sequence -> recurrent network -> unpack sequence pattern in a Module wrapped in `DataParallel`. See [My recurrent network doesn't work with data parallelism](#) section in FAQ for details.

`module (Module)` – module to be parallelized

`device_ids` (list of int or `torch.device`) – CUDA devices (default: all devices)

`output_device` (int or `torch.device`) – device location of output (default: `device_ids[0]`)

module (Module) – the module to be parallelized